Énoncé du projet

Rappel: Principe de l'autoencodeur

Un autoencodeur est un réseau de neurones composé de deux parties :

- Un **encodeur** qui compresse l'entrée (par exemple, un spectre hyperspectral de 128 dimensions) dans un espace latent de plus petite dimension.
- Un **décodeur** qui reconstitue l'entrée originale à partir de cette représentation compressée.

L'objectif est que la reconstruction soit la plus fidèle possible, au moins pour les échantillons normaux.

Entraînement de l'autoencodeur pour la détection d'anomalies

L'entraînement suit une procédure spécifique:

1. **Sélection des données d'entraînement** : On utilise **uniquement des échantillons normaux** (purs, non adultérés).
2. **Apprentissage** : Le réseau apprend à minimiser l'erreur de reconstruction sur ces seuls échantillons normaux. Concrètement, il ajuste ses poids pour que : entrée ≈ reconstruction.
3. **Que se passe-t-il après l'entraînement ?**
    - Pour un **échantillon normal** (pur) : l'autoencodeur le reconstruit bien → erreur faible.
    - Pour un **échantillon anormal** (adultéré) : l'autoencodeur ne l'a jamais vu pendant l'entraînement. Il ne sait pas le reconstruire fidèlement → erreur élevée.
4. **Détection** : On compare l'erreur de reconstruction à un **seuil**.
    - Si erreur ≤ seuil → **Normal**
    - Si erreur > seuil → **Anomalie** (adultéré)

Passage à l'apprentissage fédéré

Dans un système d'apprentissage fédéré appliqué à la détection d'anomalies, chaque client possède ses propres données. Ces données ne peuvent pas être partagées pour des raisons de confidentialité. Cependant, un simple modèle local entraîné uniquement sur les données d'un client peut manquer des anomalies qui ne deviennent visibles qu'en comparant les comportements entre plusieurs acteurs.

C'est pourquoi le framework qu'on doit implémenter utilise deux niveaux de détection :

- Une **détection locale**, réalisée par chaque client avec son propre autoencodeur et un seuil local.
- Une **détection globale**, réalisée par les clients avec le modèle global en utilisant un autoencodeur agrégé (modèle global) et un seuil commun à tous (seuil global).

Métriques d'évaluation

Evaluation - Côté client

- Exactitude, précision, rappel, F1-score

- Calculés séparément pour :
    - le modèle local (seuil local)
    - le modèle global (seuil global)

Evaluation - Côté serveur

- Métriques fédérées (Federated) : moyenne pondérée des métriques globales des clients
- Métriques centralisées (centralized) : évaluation directe sur un jeu de test détenu par le serveur

---

Spécifications techniques pour l'implémentation

Dataset

Vous travaillerez avec **un dataset quelconque**, du moment qu'il permet de faire une détection d'anomalies (deux classes : normale / anormale). Voici quelques exemples possibles :

- Dataset de détection de fraudes bancaires
- Dataset de détection de défauts industriels
- Ou tout autre dataset de votre choix

Important : Vous n'utiliserez **que les échantillons de la classe normale** pour l'entraînement de l'autoencodeur. Les échantillons anormaux ne serviront que pour le test et l'évaluation.

Framework fédéré

Vous utiliserez **Flower**, un framework Python spécialisé pour l'apprentissage fédéré. Flower permet de :

- Définir des clients qui entraînent localement leur modèle
- Communiquer avec un serveur central qui agrège les modèles
- Simuler facilement différents scénarios (nombre de clients, tours de communication, etc.)

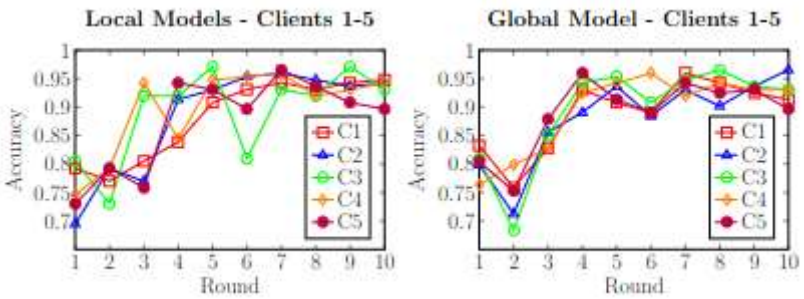
Architecture de l'autoencodeur (à adapter selon votre dataset)

- Couche d'entrée : dimension = nombre de features de votre dataset
- Couches de l'encodeur : réduction progressive (ex: 128 → 64 → 32 → 16)
- Dimension latente : faible (ex: 8 à 16)
- Couches du décodeur : expansion symétrique (ex: 16 → 32 → 64 → 128)
- Fonction de perte : Mean Squared Error (MSE)

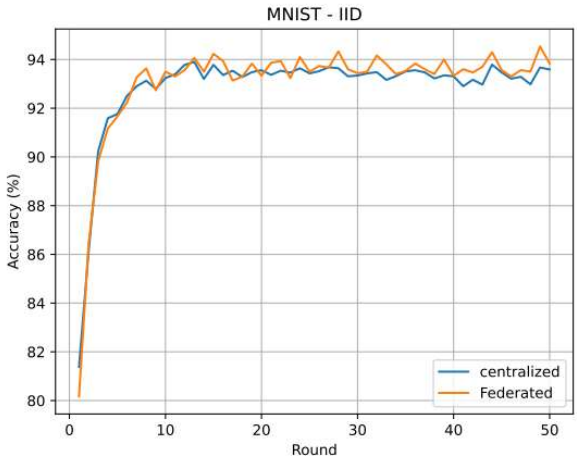
Travail à réaliser

1. **Choisir un dataset** publique connu pour une tâche de détection d'anomalies
2. **Prétraiter les données** : normalisation, séparation entraînement/test, extraction des échantillons normaux pour l'entraînement

3. **Implémenter un autoencodeur** avec TensorFlow/Keras ou PyTorch
4. **Simuler plusieurs clients** (ex: 3 à 5 clients) avec partition des données (IID ou non-IID)
5. **Implémenter le framework fédéré avec Flower** :
    - Côté client : entraînement local, calcul du seuil local, évaluation (côté client)
    - Côté serveur : agrégation des modèles (FedAvg ou FedProx), calcul du seuil global, évaluation (côté serveur).
6. **Évaluer** :
    - Comparer l'évolution des métriques au fil des rounds de communication
    - Côté Client: (exemple)



- Côté Serveur: (exemple)



Bonus possibles

- Comparer FedAvg vs FedProx sur des données non-IID
- Tester différents nombres de clients ou taux de participation
- Comparer l'autoencodeur fédéré avec un MLP fédéré supervisé ou autre baseline